C Programming — Error Analysis

Complete Answer Sheet: Problem 1 to Problem 10

Line Number | Error Type | Error Description | Correction

Problem 1: Multiple Error Types (Syntax, Semantic, Logical, Runtime)

Line	Error Type	Error Description	Correction
8	Syntax	Missing semicolon at end of: int x = 10	int x = 10;
13	Logical	Assignment used instead of comparison: if(x = 10) always sets x=10 and is always true	if(x == 10)
18	Semantic	String literal assigned to int variable: int marks = "Ninety"	int marks = 90;
20	Semantic	Undeclared variable 'z' used in printf	Declare: int z = 0; before use
22	Syntax	Literal value on left of assignment: 100 = x; — LHS must be a variable	x = 100;
26	Runtime	Array index out of bounds: arr[10] but arr is declared as arr[5]	arr[4] = 50; (valid index: 0-4)
28	Semantic	Incompatible pointer type: float *ptr = &x; where x is int	int *ptr = &x;
30	Semantic	Wrong format specifier: printf("%f", x) but x is int	printf("%d\n", x);
34	Runtime	Division by zero: a/b where b=0 causes program crash	Check b!=0 before division
36	Semantic	Multi-character literal assigned to char: char ch = 'AB'	char ch = 'A';
38	Logical	Both if and else print same message: 'X is greater' regardless of condition	else printf("Y is greater");
42	Semantic	Return value of sqrt() ignored without storing or printing result	printf("%f\n", sqrt(25));
44	Semantic	Call to undefined function: show() has no declaration or definition	Remove call or define void show() {}
46	Logical	Semicolon after while condition creates empty loop body: while(x<15); makes infinite loop or no-op	while(x < 15) { x++; } — remove the semicolon
50	Syntax	'continue' used outside a loop — invalid	Remove the continue statement

Line	Error Type	Error Description	Correction
52	Syntax	'break' used outside a loop or switch — invalid	Remove the break statement
54	Semantic	Function calculate() requires 2 arguments but called with 1: calculate(10)	int result = calculate(10, 20);

Problem 2: Syntax Errors in Control Structures

Line	Error Type	Error Description	Correction
9	Syntax	Missing semicolon: int num2 = 20	int num2 = 20;
21	Syntax	Missing closing parenthesis: if(choice == 1	if(choice == 1)
31	Syntax	Extra closing parenthesis: else if(choice == 3))	else if(choice == 3)
44	Syntax	Missing closing parenthesis in for loop: for(int i=0; i<5; i++	for(int i=0; i<5; i++)
50	Syntax	Missing closing parenthesis in while: while(choice > 0	while(choice > 0)
57	Syntax	Missing semicolon after do-while condition: while(choice > 5)	while(choice > 5);
65	Syntax	Missing colon after case label: case 2	case 2:
80	Syntax	Unclosed single quote: char grade = 'A; — missing closing quote	char grade = 'A';
84	Syntax	Missing closing bracket in array declaration: int arr[5;	int arr[5];
97	Syntax	Missing closing brace for outer block — unmatched {	Add } to close the opened block
108	Syntax	Missing semicolon: printf("End of Program\n")	printf("End of Program\n");

Problem 3: Type Mismatch and Semantic Errors

Line	Error Type	Error Description	Correction
8	Semantic	String literal assigned to int: int age = "Twenty"	int age = 20;
12	Semantic	String literal assigned to char: char grade = "A" (double quotes = string, needs single quotes)	char grade = 'A';
14	Semantic	Wrong format specifier: printf("%d", salary) but salary is float	printf("%f\n", salary);
16	Semantic	Wrong format specifier: printf("%f", marks) but marks is int	printf("%d\n", marks);

Line	Error Type	Error Description	Correction
19	Semantic	Undeclared variable 'score' used in: total = marks + score	Declare int score = 0; before use
21	Semantic	Variable 'x' declared twice in same scope: int x = 10; int x = 20;	int x = 10; x = 20; — remove second declaration
23	Syntax	Literal on left side of assignment: 100 = x	x = 100;
26	Semantic	Wrong format specifier: printf("%s", num) but num is int	printf("%d\n", num);
30	Semantic	Wrong format specifier: printf("%d", pi) but pi is float	printf("%f\n", pi);
33	Semantic	Float used as array index: arr[2.5] — index must be integer	arr[2] = 100;
37	Semantic	Too few arguments: result = add(10) — add() needs 2 arguments	result = add(10, 20);
41	Semantic	Too few arguments: avg = average(10,20) — average() needs 3 arguments	avg = average(10, 20, 30);
43	Semantic	String literal assigned to int: int temperature = "35"	int temperature = 35;
46	Semantic	String literal assigned to float: float discount = "10.5"	float discount = 10.5;
50	Semantic	Direct assignment to char array: name = "Robil" — use strcpy()	strcpy(name, "Robil");
54	Semantic	Undeclared variable 'totalMarks': count = totalMarks + 10	Declare int totalMarks = 0; before use
63	Semantic	Too many arguments: sum = add(a,b,c) — add() only takes 2 params	sum = add(a, b);
72	Logical	Assignment used as condition: if(studentMarks = 100) always sets to 100 and is always true	if(studentMarks == 100)
74	Semantic	String literal assigned to int: int size = "Large"	int size = 5;
78	Semantic	Float printed with %d: printf("%d", percentage) but percentage is float	printf("%f\n", percentage);
81	Semantic	String used as array index: arr2["five"] — must be integer	arr2[5] = 100;
90	Semantic	String used as case label: case "2" — switch cases must be integer constants	case 2:
95	Semantic	Undeclared function: multiply() never declared or defined	Define multiply() or use add()
103	Semantic	Float variable printed with %d: printf("%d", areaRect) but areaRect is float	printf("%f\n", areaRect);

Line	Error Type	Error Description	Correction
113	Semantic	Undeclared variable 'value': printf("%d", value)	Declare int value = 0; before use
115	Semantic	String literal assigned to int: int employeeId = "EMP101"	Use char employeeId[10] = "EMP101";

Problem 4: Missing Function Definitions (Linker / Semantic Errors)

Line	Error Type	Error Description	Correction
38	Semantic	extern float companyProfit declared but never defined anywhere in file	Add: float companyProfit = 0.0; as global definition
55	Semantic	Call to calculateSalary(50000) but definition is missing (void calculateSalary(float) declared but never defined)	Add definition: void calculateSalary(float basic){ printf("Salary=%.2f\n",basic); }
57	Semantic	Call to printStudentDetails() but definition is missing (declared but never defined)	Add definition: void printStudentDetails(){ printf("Student Details\n"); }
63	Semantic	Call to showResult() but definition is missing (declared but never defined)	Add definition: void showResult(){ printf("Result shown\n"); }
67	Semantic	Call to calculateAverage() but definition is missing (declared but never defined)	Add definition: float calculateAverage(int a,int b,int c){ return (a+b+c)/3.0f; }
73	Semantic	Call to generateReport() — not declared or defined anywhere	Declare and define void generateReport(){...}
75	Semantic	Call to displayEmployee() — not declared or defined anywhere	Declare and define void displayEmployee(){...}
77	Semantic	Call to processData() — not declared or defined anywhere	Declare and define void processData(){...}
79	Semantic	Call to saveRecord() — not declared or defined anywhere	Declare and define void saveRecord(){...}
81	Semantic	Call to loadRecord() — not declared or defined anywhere	Declare and define void loadRecord(){...}
83	Semantic	Call to printInvoice() — not declared or defined anywhere	Declare and define void printInvoice(){...}
85	Semantic	Call to calculateTax() — not declared or defined anywhere	Declare and define void calculateTax(){...}
87	Semantic	Call to validateUser() — not declared or defined anywhere	Declare and define void validateUser(){...}
89	Semantic	Call to sendNotification() — not declared or defined anywhere	Declare and define void sendNotification(){...}
91	Semantic	Call to generateCertificate() — not declared or defined anywhere	Declare and define void generateCertificate(){...}

Problem 5: Runtime Errors (Memory, Pointers, Division)

Line	Error Type	Error Description	Correction
12	Runtime	Division by zero: a/b where b=0 causes undefined behavior / crash	if(b != 0) printf("%d\n", a/b);
14	Runtime	Null pointer dereference: ptr1=NULL then *ptr1=50 causes segfault	Allocate memory first: ptr1 = malloc(sizeof(int)); *ptr1 = 50;
18	Runtime	Array out of bounds: arr[10]=100 but arr declared as arr[5]	arr[4] = 100; // valid index 0-4
21	Runtime	Uninitialized pointer dereference: *ptr2=200 without allocating memory	int *ptr2 = malloc(sizeof(int)); *ptr2 = 200;
25	Runtime	Buffer overflow: strcpy(name, "Programming") into char name[5] — too small	char name[20]; strcpy(name, "Programming");
29	Runtime	Uninitialized pointer read: printf("%d", *ptr3) — ptr3 not initialized	int *ptr3 = malloc(sizeof(int)); *ptr3 = 0;
33	Runtime	File pointer not checked: fopen may return NULL if file not found; fscanf on NULL crashes	if(fp != NULL){ fscanf(fp,"%d",&a;); fclose(fp); }
37	Runtime	Heap buffer overflow: malloc(5) gives 5 bytes but strcpy(ptr4,"Computer Science") needs 17 bytes	char *ptr4 = malloc(20); strcpy(ptr4,"Computer Science");
41	Runtime	Use after free: printf(ptr4[0]) after free(ptr4) — dangling pointer	Remove printf after free; set ptr4=NULL after free
44	Runtime	Double free: free(ptr5) called twice — causes heap corruption	Remove second free(ptr5); set ptr5=NULL after first free
50	Runtime	Modulo by zero: x%y where y=0 causes undefined behavior	if(y != 0) result = x % y;
54	Runtime	Buffer overflow in strcat: str1[10] has only 5 bytes free but appending 16-char string	char str1[30] = "Hello"; strcat(str1,"WorldProgramming");
59	Runtime	Heap overflow in loop: malloc(3*sizeof(int)) but writing numbers[0..9] — 10 elements into 3-int space	numbers = malloc(10 * sizeof(int)); for(int i=0;i<10;i++)
64	Runtime	Array out of bounds: matrix[5][5]=100 but matrix declared as [3][3]	matrix[2][2] = 100;
67	Runtime	Null pointer dereference: ptr6=NULL then printf(ptr6[0]) — segfault	int val=0; printf("%d\n", val);
70	Runtime	Unsafe function gets(): no bounds checking — buffer overflow risk	fgets(password, sizeof(password), stdin);
76	Runtime	Use after free: *ptr7=1000 after free(ptr7) — dangling pointer write	Do not write to ptr7 after free; set ptr7=NULL
79	Runtime	Uninitialized variable: int age; used in printf without being set	int age = 0; or read with scanf

Line	Error Type	Error Description	Correction
82	Runtime	Negative array index: arr[index] where index=-1 — out of bounds	Ensure index >= 0 before use
85	Runtime	Uninitialized pointer dereference: if(*ptr8>0) without ptr8 being set	int *ptr8 = malloc(sizeof(int)); *ptr8 = 0;
88	Runtime	Null pointer passed to printf %s: char *text=NULL then printf("%s",text)	printf("%s\n", text ? text : "(null)");
92	Runtime	Huge malloc then immediate use: malloc(1000000000*sizeof(int)) may fail; no NULL check before use	hugeArray=malloc(size*sizeof(int)); if(hugeArray){hugeArray[0]=1; free(hugeArray);}
98	Runtime	Infinite loop: while(1){ ptr9++; } increments NULL pointer forever — crash	Remove infinite loop or add termination condition

---

Problem 6: Logical Errors

Line	Error Type	Error Description	Correction
12	Logical	Even/Odd check reversed: num%2==1 prints 'Even' — should print Odd	if(num%2==0) printf("Even"); else printf("Odd");
20	Logical	Grade ladder reversed: marks>=90 prints Grade B, marks>=80 prints Grade A — A and B swapped	if(marks>=90) Grade A; else if(marks>=80) Grade B; ...
29	Logical	Incomplete leap year check: only year%4==0 tested; century years (1900) would be wrongly called leap	if((year%4==0 && year%100!=0) || year%400==0) Leap Year;
36	Logical	Voting eligibility reversed: age>18 prints 'Not Eligible'; should be eligible when age>=18	if(age >= 18) printf("Eligible"); else printf("Not Eligible");
43	Logical	Finding smallest instead of largest: condition a>b prints a as 'Smaller' — logic inverted	if(a > b) printf("Larger = %d", a); else printf("Larger = %d", b);
50	Logical	Finding smallest instead of largest: if(y < largest) largest=y replaces largest with smaller value	if(y > largest) largest=y; if(z > largest) largest=z;
57	Logical	Integer division loses decimal: average = totalMarks/subjects; both int so result is 83 not 83.33	average = (float)totalMarks / subjects;
63	Logical	Wrong formula: area = 2*3.14*r*r is surface area, not circle area (pi*r^2)	area = 3.14 * radius * radius;
70	Logical	Bonus percentages reversed: salary>30000 gets 2% (should get higher bonus), else gets 20%	if(salary > 30000) bonus = salary*0.20; else bonus = salary*0.02;
78	Logical	Switch month numbers shifted: case 1 prints February instead of January, all months off by one	case 1: January; case 2: February; case 3: March; case 4: April;
91	Logical	isPrime set to 1 (not prime) inside loop when divisor found — should be 0 (not prime)	isPrime = 0; // inside the if(n%i==0) block

Line	Error Type	Error Description	Correction
95	Logical	if(isPrime) prints 'Not Prime' — condition and messages are inverted	if(isPrime) printf("Prime"); else printf("Not Prime");
102	Logical	Discount added instead of subtracted: finalAmount = purchaseAmount + discount	finalAmount = purchaseAmount - (purchaseAmount * discount / 100);
109	Logical	Assignment instead of accumulation: sum = i should be sum += i to add all values	sum += i;
115	Logical	Temperature condition reversed: temp<40 prints 'Cold Weather' — 35 is hot, not cold	if(temperature >= 40) printf("Hot"); else printf("Cold");
120	Logical	Grade message reversed: grade=='A' prints 'Average Performance' — A is Excellent	if(grade=='A') printf("Excellent"); else printf("Average");
126	Logical	Wrong formula: rectangleArea = length + width; should be multiplication	rectangleArea = length * width;
132	Logical	Pass/Fail reversed: percentage>=60 prints 'Fail'; should be Pass	if(percentage >= 60) printf("Pass"); else printf("Fail");

Problem 7: Array and String Errors

Line	Error Type	Error Description	Correction
10	Runtime	Array out of bounds: for i<=5 accesses marks[5] which is out of range for int marks[5]	for(int i=0; i<5; i++)
22	Logical	Integer division: average = total/5 loses decimal part since total and 5 are both int	average = total / 5.0f;
27	Runtime	Off-by-one in assignment loop: for i<=10 writes to numbers[10] — out of bounds for int numbers[10]	for(int i=0; i<10; i++) numbers[i] = (i+1)*10;
33	Runtime	Buffer overflow: strcpy(name, "ComputerScience") — 15 chars into char name[10]	char name[20]; strcpy(name, "ComputerScience");
38	Semantic	Direct string assignment to char array: city = "Delhi" — not allowed after declaration	strcpy(city, "Delhi");
43	Logical	String comparison using == compares pointers not content: str1==str2 always false	if(strcmp(str1, str2) == 0)
49	Runtime	Unsafe gets(): no buffer size limit — buffer overflow risk for char password[8]	fgets(password, sizeof(password), stdin);
54	Runtime	strcat without space: firstName[20]="Robil"(5 chars) + lastName(8 chars)=13 chars — no space between names	strcat(firstName, " "); strcat(firstName, lastName);
59	Runtime	char grade[5] printed with %s without null terminator — may print garbage	grade[2] = '\0'; printf("%s\n", grade);

Line	Error Type	Error Description	Correction
63	Runtime	Negative index access: arr[-1] — undefined behavior / out of bounds	Remove or use valid index (0 to 4)
72	Runtime	Buffer overflow: char email[15] but "student@gmail.com" is 17 chars	char email[25] = "student@gmail.com";
77	Runtime	Write and access beyond declared size: scores[3]=60 — scores declared as scores[3] (indices 0-2)	int scores[4]; scores[3]=60;
82	Runtime	Buffer overflow: char subject[10] = "Programming" — 11 chars + null = 12 bytes, doesn't fit in 10	char subject[15] = "Programming";
87	Logical	String comparison with == compares addresses not content: username=="admin" never true	if(strcmp(username, "admin") == 0)
93	Runtime	Buffer overflow: strcpy(mobile, "987654321012") — 12 digits + null = 13 bytes into char mobile[10]	char mobile[15]; strcpy(mobile, "987654321012");
98	Runtime	Off-by-one in loop: for i<=5 accesses attendance[5] — out of bounds for int attendance[5]	for(int i=0; i<5; i++)
104	Runtime	Out of bounds access: course[20] accesses index 20 which is past the end of char course[20] (valid: 0-19)	printf("%c\n", course[0]); // or valid index
109	Runtime	char college[20] printed with %s but no null terminator added — may print garbage	college[5] = '\0'; printf("%s\n", college);
116	Runtime	char section[3] = "IT-A" — "IT-A" needs 5 bytes (4 chars + null) but only 3 allocated	char section[6] = "IT-A";

Problem 8: Function Errors (Arguments, Logic, Return)

Line	Error Type	Error Description	Correction
15	Semantic	Too few arguments: result = add(num1) — add() requires 2 parameters	result = add(num1, num2);
20	Semantic	Too few arguments: avg = average(70,80) — average() requires 3 parameters	avg = average(70, 80, 90);
44	Logical	swap(x,y) passes by value — values of x,y in main() are NOT swapped	Use pointers: swap(&x, &y); and change void swap(int *a, int *b)
55	Semantic	Call to undeclared function: calculateGrade(marks) not declared or defined	Declare and define: void calculateGrade(int m){...}
60	Semantic	Call to undeclared function: circleArea(5) not declared or defined	Declare and define: int circleArea(int r){return 3.14*r*r;}
65	Semantic	Call to undeclared function: sumArray() not declared or defined	Declare and define: int sumArray(){...}

Line	Error Type	Error Description	Correction
69	Semantic	Call to undeclared function: displayResult() not declared or defined	Declare and define: void displayResult(){...}
75	Logical	factorial base case wrong: if(n==0) return 0; — should return 1 (0!=1)	if(n == 0) return 1;
85	Logical	maximum() returns wrong value: if(a>b) return b; — returns smaller, not larger	if(a > b) return a; else return b;
93	Logical	fibonacci() infinite recursion: fibonacci(n)+fibonacci(n-1) — first call never decrements	return fibonacci(n-1) + fibonacci(n-2);
104	Logical	calculatePercentage() uses int division: marks/total*100 loses precision (e.g. 70/100=0)	return (marks * 100) / total;
111	Logical	simpleInterest() returns nothing: return; but function declared as float	return si;
120	Logical	power() loop: for i=1; i<exp misses last iteration — should be i<=exp	for(int i=1; i<=exp; i++) result *= base;
129	Logical	reverseNumber() returns nothing: function is int but missing return statement	return rev; // at end of function
142	Logical	isPrime() logic inverted: returns 1 when divisor found (not prime), returns 0 if prime	if(n%i==0) return 0; // not prime; return 1 at end
148	Logical	updateValue() passes by value: x=500 inside function doesn't affect caller's variable	void updateValue(int *x){ *x = 500; } and call: updateValue(&var);
155	Runtime	printArray loop: for i<=size accesses arr[size] — out of bounds	for(int i=0; i<size; i++)
161	Runtime	recursiveDemo() has no base case: infinitely recurses causing stack overflow	Add base: if(n>=10) return n; else return recursiveDemo(n+1);
170	Logical	rectangleArea() uses addition instead of multiplication: length+width	return length * width;
176	Logical	cube() function calculates square not cube: returns n*n instead of n*n*n	return n * n * n;

Problem 9: Pointer Errors (Memory, Null, Dangling)

Line	Error Type	Error Description	Correction
13	Runtime	Uninitialized pointer dereference: printf(*ptr1) without ptr1 pointing anywhere	int *ptr1 = malloc(sizeof(int)); *ptr1 = 0;
17	Runtime	Null pointer dereference: ptr2=NULL then *ptr2=100 causes segfault	int *ptr2 = malloc(sizeof(int)); *ptr2 = 100;
23	Runtime	After ptr3=&x; doing ptr3++ moves ptr3 to unknown memory; dereferencing it is undefined	Remove ptr3++; or use array for pointer arithmetic

Line	Error Type	Error Description	Correction
28	Runtime	Out-of-bounds pointer: ptr4+10 goes 10 elements past start of arr[5] — undefined behavior	printf("%d\n", *(ptr4 + 4)); // last valid element
35	Runtime	Use after free: printf(*ptr5) after free(ptr5) — dangling pointer read	Remove printf after free; set ptr5=NULL after free
39	Runtime	Memory leak: ptr6 = malloc then ptr6=NULL without freeing — memory is lost	free(ptr6); ptr6 = NULL;
43	Runtime	Heap overflow: malloc(5) allocates 5 bytes but loop writes 10 ints (40 bytes)	ptr7 = malloc(10 * sizeof(int));
50	Runtime	Write beyond allocated memory: ptr8[3]=40 when only 3 ints allocated (indices 0-2)	ptr8 = malloc(4 * sizeof(int)); ptr8[3]=40;
56	Runtime	Double free: free(ptr9) called twice — heap corruption	After first free: ptr9 = NULL; and remove second free
61	Runtime	Uninitialized memory read: malloc doesn't zero memory; printf(*ptr10) prints garbage	Use calloc(1,sizeof(int)) instead or set *ptr10=0 after malloc
65	Runtime	Off-by-one in loop: for i<=5 writes numbers[5] — out of bounds for 5-element malloc	for(int i=0; i<5; i++)
76	Logical	swap() swaps local pointer variables not pointed values — a,b in caller unchanged	void swap(int *a,int *b){int t=*a; *a=*b; *b=t;}
81	Semantic	pptr is int** but assigned &value where value is int — type mismatch (need int**)	int *vptr = &value; pptr = &vptr; printf("%d\n", **pptr);
85	Semantic	strcpy without including string.h — undefined behavior; also malloc(5) too small for "Programming"	#include<string.h> name=malloc(15); strcpy(name,"Programming");
89	Runtime	ptr11++ after malloc moves pointer away from allocated block; *ptr11 is now unknown memory	Remove ptr11++; print *ptr11 before incrementing
93	Runtime	Null pointer dereference: ptr12=NULL then if(*ptr12>0) causes segfault	if(ptr12 != NULL && *ptr12 > 0)
99	Runtime	Off-by-one: ptr13[100]=500 when malloc(100*sizeof(int)) — valid indices 0-99	ptr13[99] = 500;
104	Runtime	Use after free: *ptr14=999 after free(ptr14) — dangling pointer write	Do not write to ptr14 after free; set ptr14=NULL
108	Semantic	Assigning int(*)[3] to int*: p=matrix — incompatible pointer types	int *p = &matrix[0][0]; // flatten 2D to 1D pointer
111	Runtime	*(p+20) goes past matrix[3][3] (9 elements) — out of bounds access	*(p+8) // last valid element of 3x3 matrix
114	Runtime	Memory leak: ptr15 malloc'd twice without freeing first allocation	free(ptr15); ptr15 = malloc(sizeof(int)); // free before reallocating

Line	Error Type	Error Description	Correction
67	Runtime	printArray loop: for i<=size prints arr[size] — one past end, out of bounds	for(int i=0; i<size; i++)

Problem 10: Combined Errors (All Types)

Line	Error Type	Error Description	Correction
8	Syntax	Missing semicolon: int choice = 1	int choice = 1;
10	Semantic	String literal assigned to int: int marks = "90"	int marks = 90;
16	Semantic	Too few arguments: result = add(a) — add() requires 2 arguments	result = add(a, b);
20	Logical	Assignment in condition: if(a = b) sets a=b (always true), not comparing	if(a == b)
24	Semantic	Undeclared variable 'score' used in printf	Declare int score = 0; before use
27	Runtime	Array out of bounds: arr[10] when arr declared as arr[5]	arr[4] = 100;
29	Runtime	Negative index: arr[-1] is out of bounds — undefined behavior	Use valid index: arr[0]
32	Runtime	Buffer overflow: strcpy(name, "ProgrammingLanguage") — 19 chars into char name[10]	char name[25]; strcpy(name, "ProgrammingLanguage");
36	Semantic	Direct string assignment to char array: city = "Delhi" — illegal after declaration	strcpy(city, "Delhi");
40	Logical	String comparison using ==: user=="admin" compares pointers not content — always false	if(strcmp(user, "admin") == 0)
46	Runtime	Division by zero: x/y where y=0 — crash	if(y != 0) printf("%d\n", x/y);
49	Runtime	Null pointer dereference: ptr1=NULL then *ptr1=500 — segfault	ptr1 = malloc(sizeof(int)); *ptr1 = 500;
52	Runtime	Uninitialized pointer dereference: printf(*ptr2) without ptr2 being set	int *ptr2 = malloc(sizeof(int)); *ptr2 = 0;
57	Runtime	Use after free: printf(*ptr3) after free(ptr3) — dangling pointer	Print *ptr3 BEFORE free; after free set ptr3=NULL
59	Runtime	Double free: free(ptr3) called twice — heap corruption	Remove second free; set ptr3=NULL after first
62	Runtime	Heap overflow: malloc(5 bytes) but loop writes 10 ints into ptr4	ptr4 = malloc(10 * sizeof(int));
67	Runtime	Memory leak: ptr5 malloc'd twice without freeing first allocation	free(ptr5); ptr5 = malloc(sizeof(int));
70	Semantic	Too few arguments: avg = average(70,80) — average() takes 3 parameters	avg = average(70, 80, 90);

Line	Error Type	Error Description	Correction
75	Logical	Voting eligibility wrong: age>18 should be age>=18 to include 18-year-olds	if(age >= 18) printf("Eligible");
83	Logical	isPrime set to 1 when divisor found — should be 0; message also inverted	isPrime = 0; // inside loop; then if(isPrime) print Prime else Not Prime
88	Semantic	Call to undeclared function: displayReport() not declared in this file	Add definition or include header where displayReport is defined
90	Semantic	Call to undeclared function: calculateSalary() — never declared or defined	Declare and define void calculateSalary(){...}
92	Semantic	extern int totalStudents declared but definition missing in linked file	Add: int totalStudents = 500; as global definition
95	Runtime	fopen without NULL check: fscanf on NULL fp (file not found) causes crash	fp=fopen(...); if(fp!=NULL){ fscanf(...); fclose(fp); }
99	Syntax	Missing colon after case label: case 1	case 1:
102	Logical	case 2 missing break — falls through to default, printing both 'Two' and 'Other'	case 2: printf("Two\n"); break;
108	Semantic	Multi-character literal: char grade = 'AB' — char holds only one character	char grade = 'A';
113	Runtime	Write beyond array: scores[3]=40 when scores declared as scores[3] (indices 0-2)	int scores[4]; scores[3]=40;
118	Runtime	Loop condition i<=3 accesses scores[3] — out of bounds for scores[3]	for(int i=0; i<3; i++)
124	Logical	Wrong formula: area = 2*3.14*r*r is surface area formula, not circle area	area = 3.14f * radius * radius;
128	Logical	Semicolon after while: while(choice>0); creates infinite empty loop (choice never decrements)	while(choice > 0) { choice--; } — remove semicolon
132	Syntax	'continue' used outside a loop — invalid syntax	Remove the continue statement